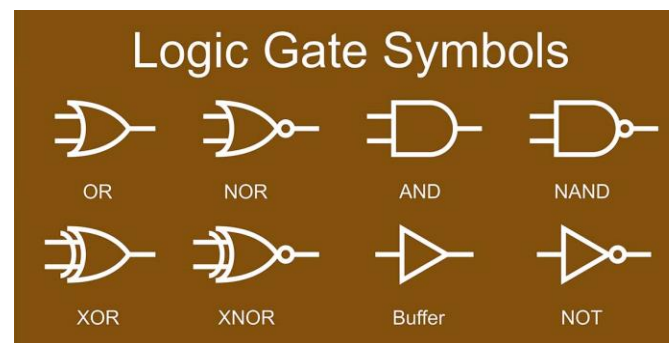
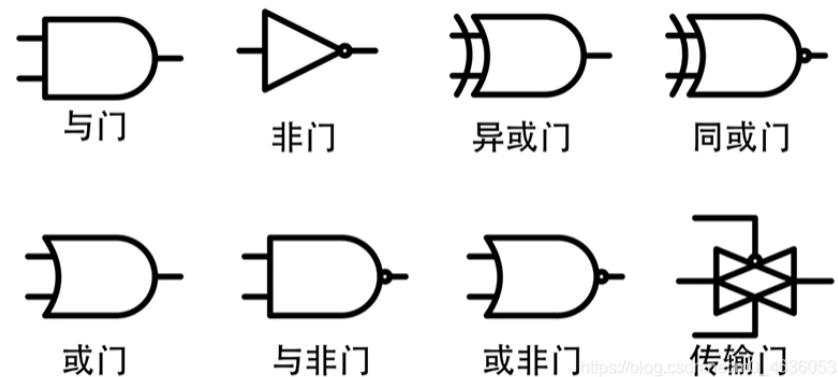




第三章 逻辑函数及其简化

周玲

ling.zhou@seu.edu.cn

内容

○ 3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

○ 3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

○ 3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

○ 3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

○ 3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

○ 3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

○ 3.7 逻辑函数的简化

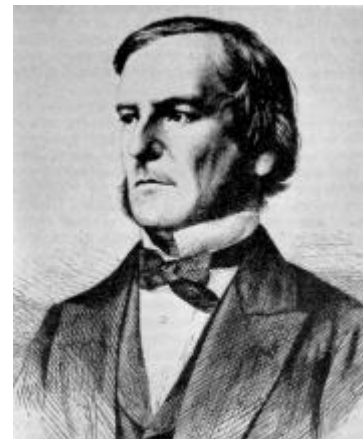
- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

○ 本章小结

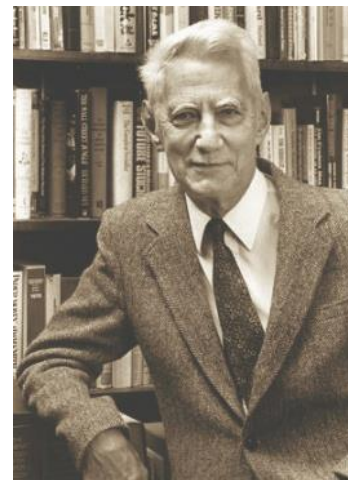
介绍

谁家学霸两百年：从布尔代数到人工智能
百科TA说 (baidu.com)

- 逻辑代数 (logic algebra) 由英国数学家乔治·布尔于1854年创建，也称布尔代数 (Boolean algebra) 。
 - 1847年，布尔出版了《逻辑的数学分析》 (The Mathematical Analysis of Logic)
 - 1854年，他出版了《思维规律的研究》， (An Investigation of The Laws of Thought) 这是他最著名的著作。
 - 1859年《微分方程》
- 1938年克劳德·埃尔伍德·香农将布尔代数应用到开关和继电器网络的分析和设计，又成为开关代数。
- 本章主要介绍逻辑代数的基本知识，介绍逻辑函数及其描述方法和逻辑函数的简化。



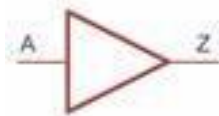
George Boole
1815.11.2~1864
19世纪最重要的数学家之一



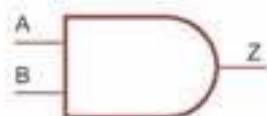
Claude Elwood Shannon
1916.4.30 — 2001.2.24
美国数学家、信息论的创始人

基本逻辑运算

- 逻辑代数是用来处理命题之间的逻辑关系的代数系统。
 - 命题：表示为逻辑变量
 - 命题间的逻辑关系：逻辑函数表示
 - 逻辑函数：由逻辑变量经过逻辑运算产生。
- 基本逻辑运算：
 - 非（NOT）、与（AND）、或（OR）运算



BUF	A	Z
	0	0
	1	1



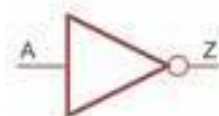
AND	A	B	Z
	0	0	0
	0	1	0
	1	0	0
	1	1	1



OR	A	B	Z
	0	0	0
	0	1	1
	1	0	1
	1	1	1



XOR	A	B	Z
	0	0	0
	0	1	1
	1	0	1
	1	1	0



INV	A	Z
	0	1
	1	0



NAND	A	B	Z
	0	0	1
	0	1	1
	1	0	1
	1	1	0



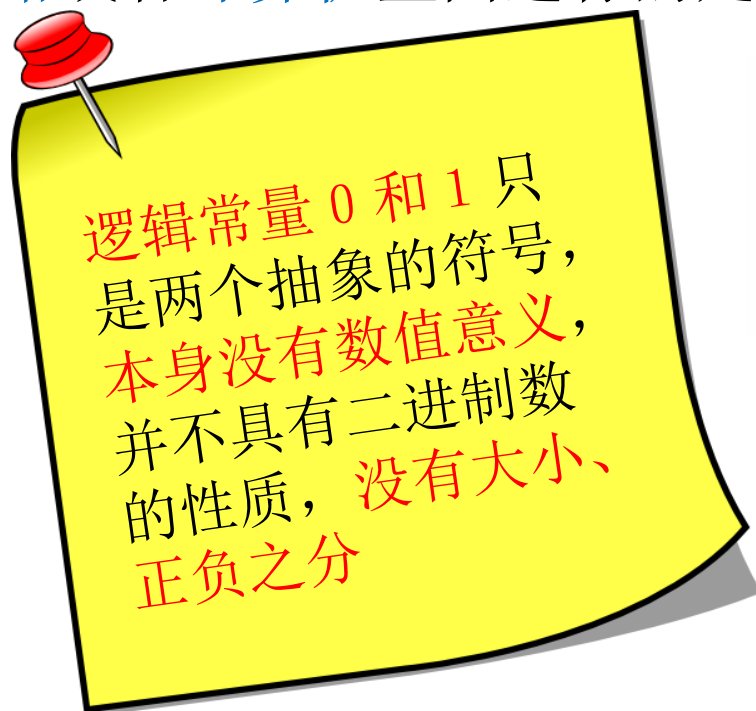
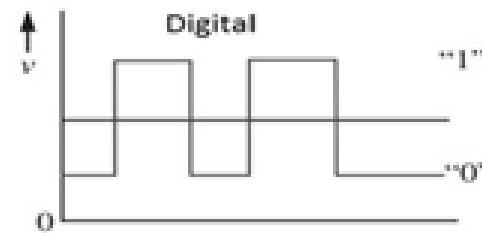
NOR	A	B	Z
	0	0	1
	0	1	0
	1	0	0
	1	1	0



XNOR	A	B	Z
	0	0	1
	0	1	0
	1	0	0
	1	1	1

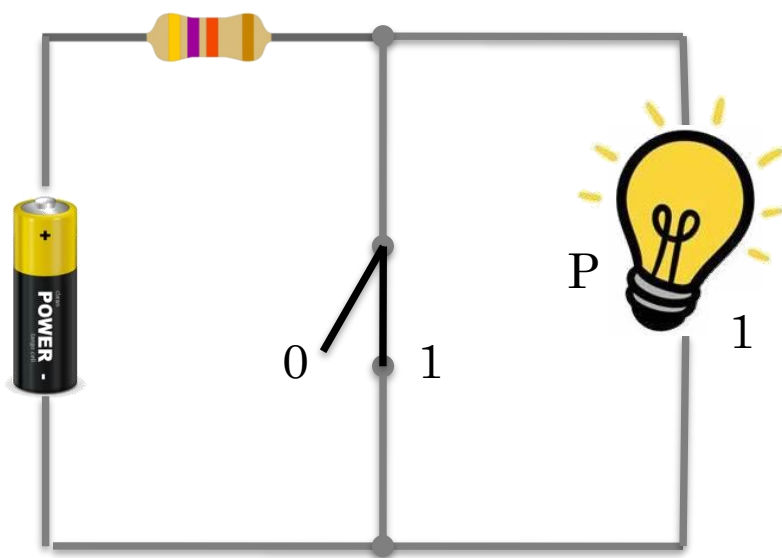
逻辑代数的二值逻辑

- 二值逻辑：任何逻辑命题只有True和False两个可能取值，没有“似是而非”的模糊空间。
 - 逻辑真值：1 代表  0 代表 
- 在数字电路或者计算机里面进行的是二进制逻辑运算。



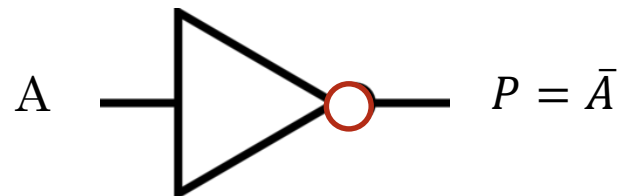
逻辑非和非运算

逻辑非电路实例

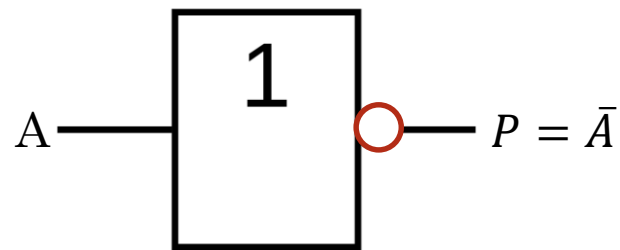


Input A	Output $P = \text{not } A$
0	1
1	0

逻辑非门 (**NOT gate**)

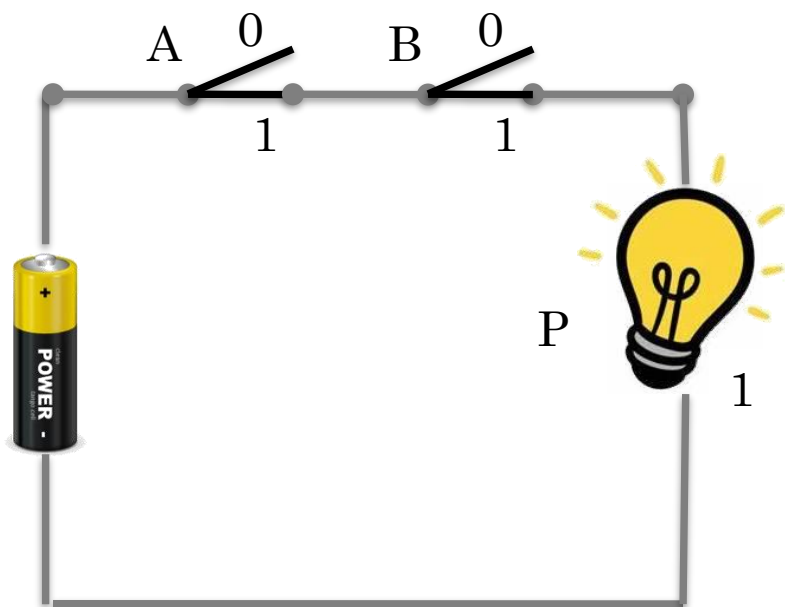


美标：美国MIL-STD-806B为欧美书刊文和EDA软件所广泛采用



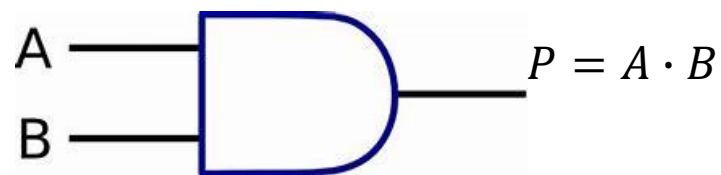
国标：符合我国国标GB4728.12-85标准
国际电工委员会
在IEC617-12中推荐使用的标准符号

逻辑乘和与运算

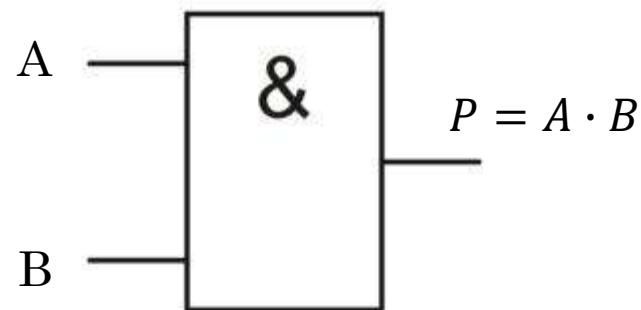


Input		Output
A	B	$P = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

逻辑与门 (AND gate)



美国MIL-STD-806B



国标GB4728.12-85

与逻辑运算规则

$$A \cdot 0 = 0$$

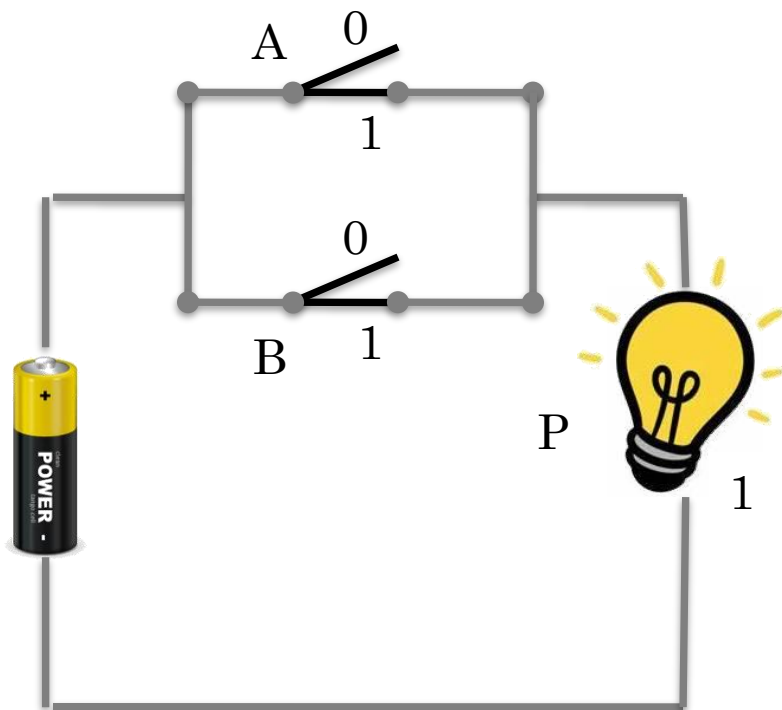
$$A \cdot 1 = A$$

$$1 \cdot A = A$$

$$A \cdot \bar{A} = 0$$

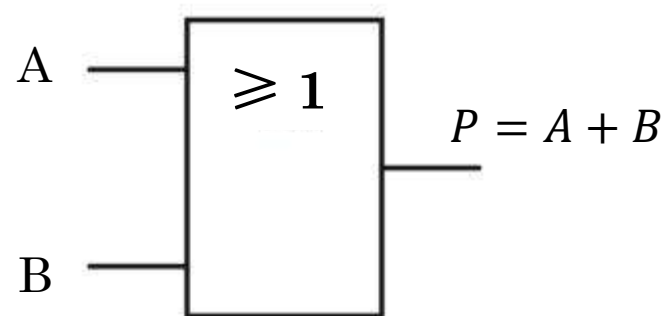
$$A \cdot A = A$$

逻辑加和或运算



Input		Output
A	B	$P = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

逻辑或门 (OR gate)



或逻辑运算规则

$$A + 0 = A$$

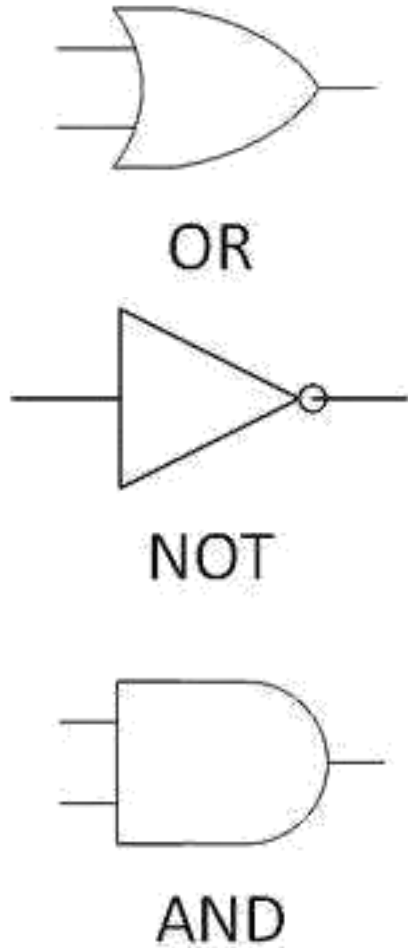
$$A + 1 = 1$$

$$1 + A = 1$$

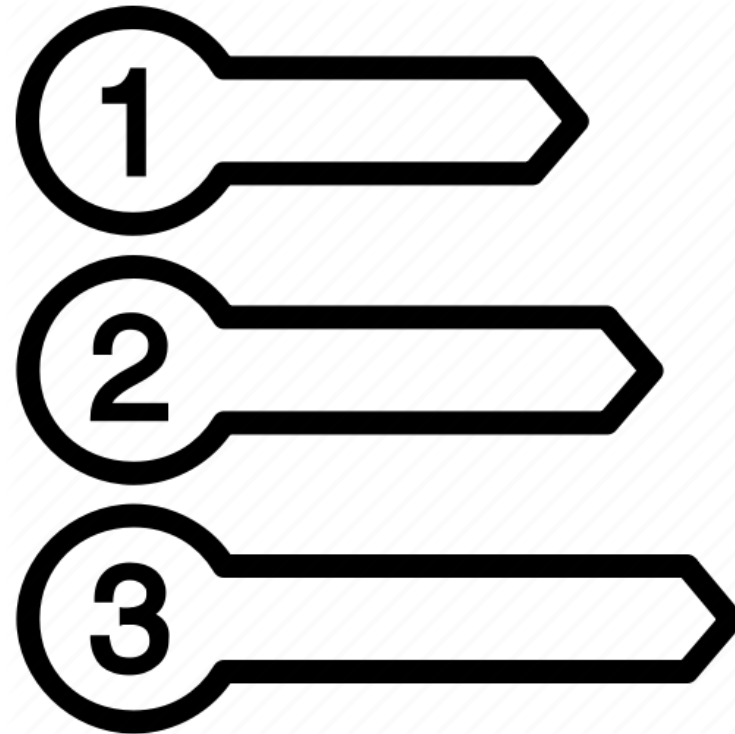
$$A + A = A$$

$$A + \bar{A} = 1$$

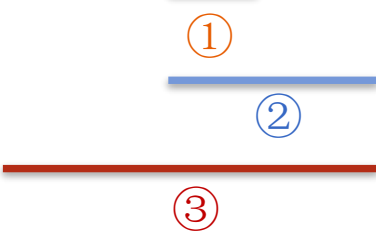
逻辑运算的优先级 (1)



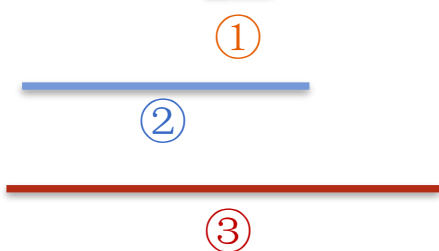
请排序



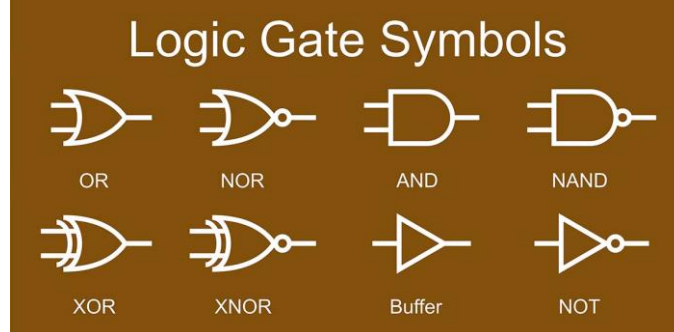
逻辑运算的优先级 (2)

$$F = A + \bar{B} \cdot C$$


若需要更改运算次序，
可以通过加括号实现

$$F = (A + \bar{B}) \cdot C$$


内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

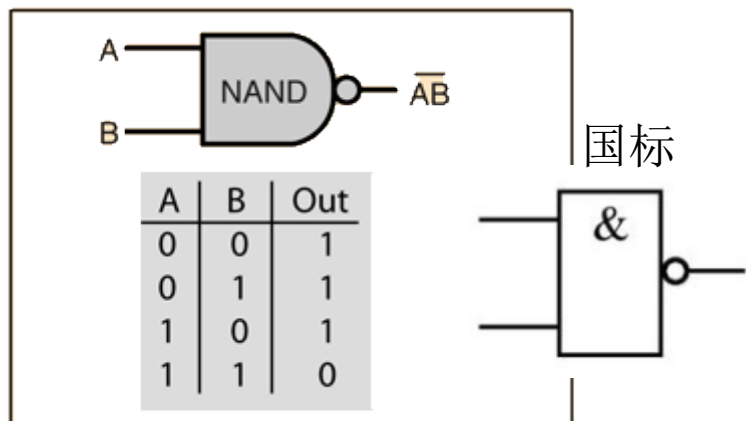
3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

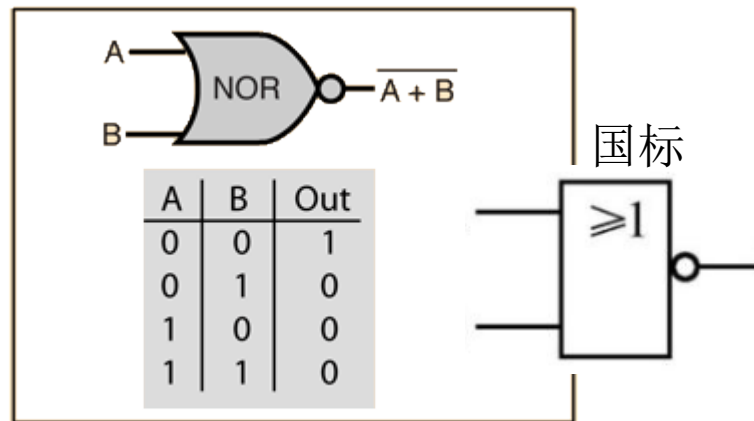
本章小结

复合逻辑运算

与非门（美标）

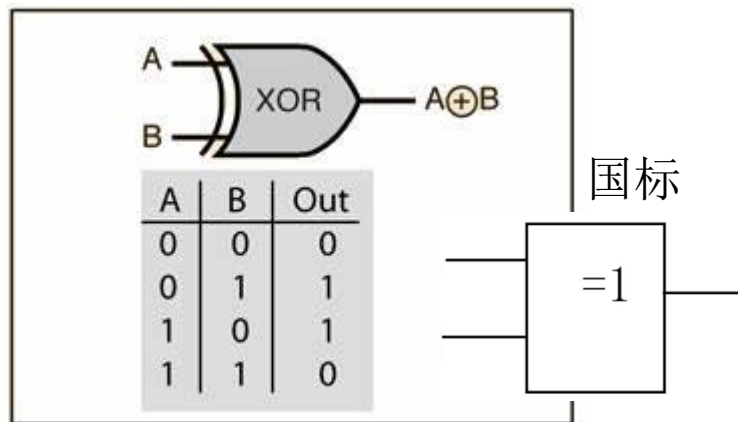


或非门（美标）



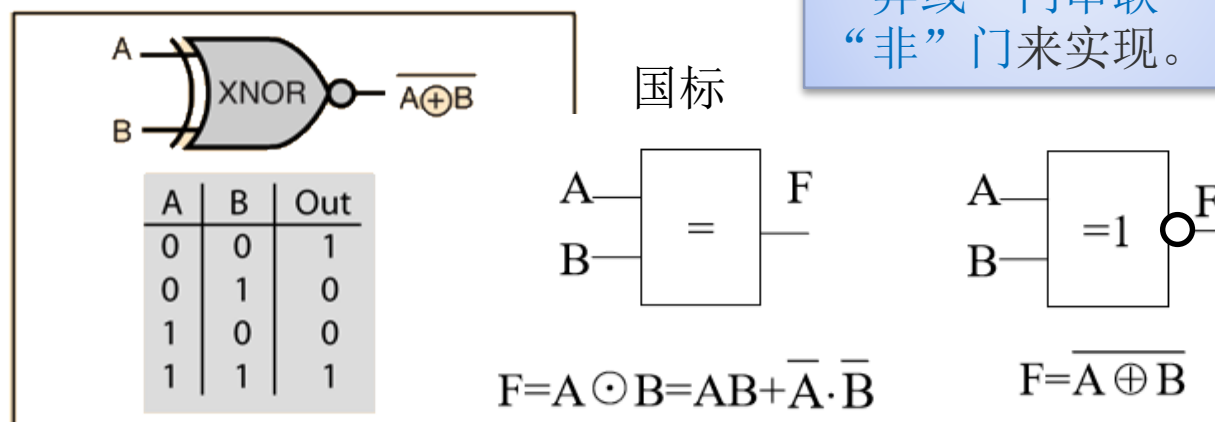
异或门（美标）

$$A \oplus B = A\bar{B} + \bar{A}B$$



同或门（美标）

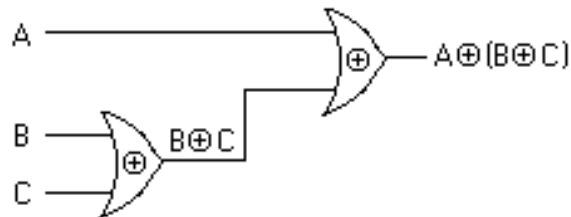
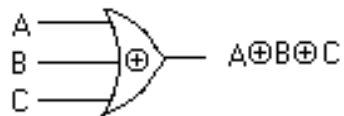
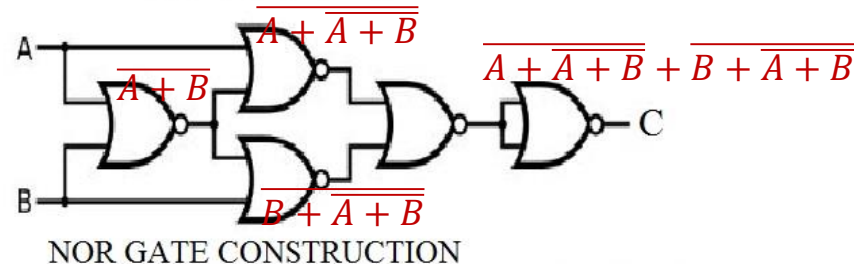
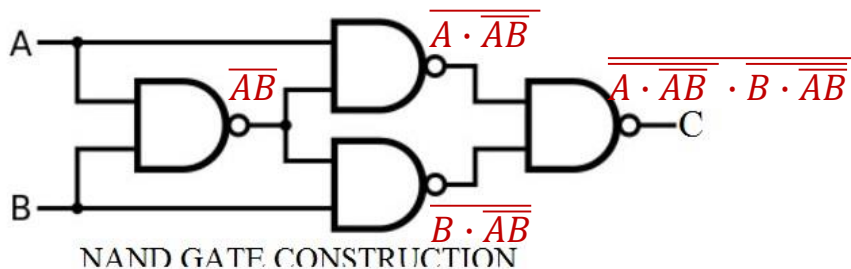
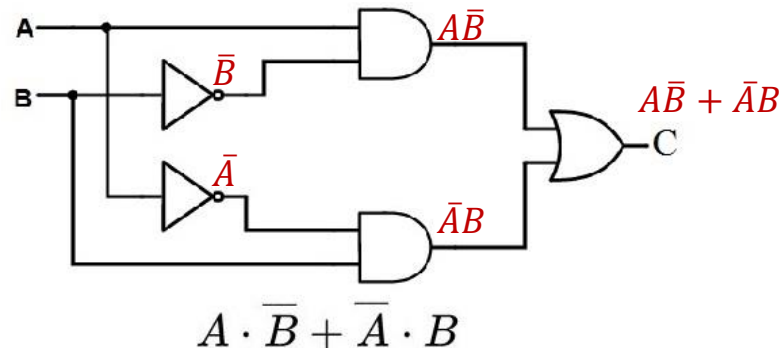
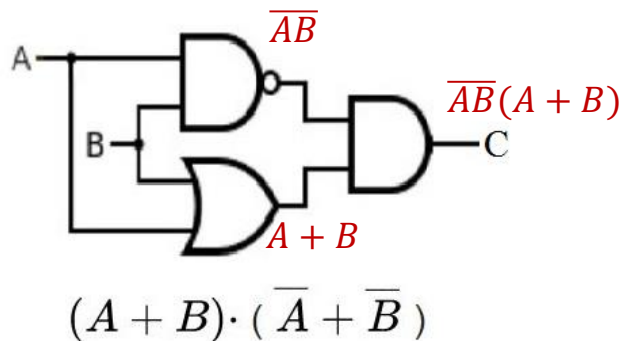
$$F = \overline{A \oplus B} = A \odot B = AB + \bar{A}\bar{B}$$



电路中较少使用“同或” XNOR 门，并且“同或”门可用“异或”门串联一个“非”门来实现。

eXclusive-OR gate

异或门 XOR GATE CIRCUIT CONFIGURATIONS

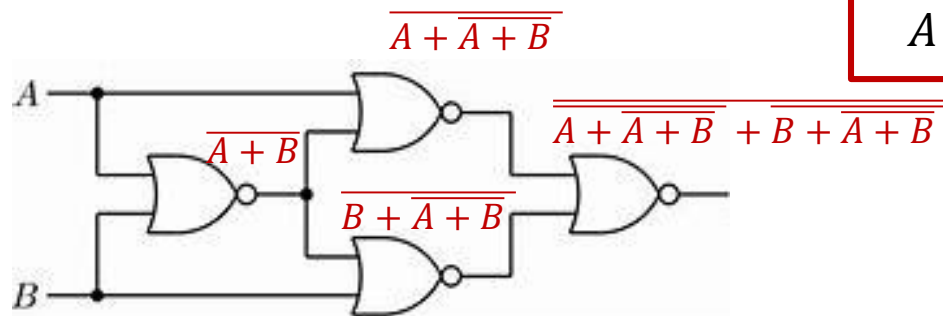
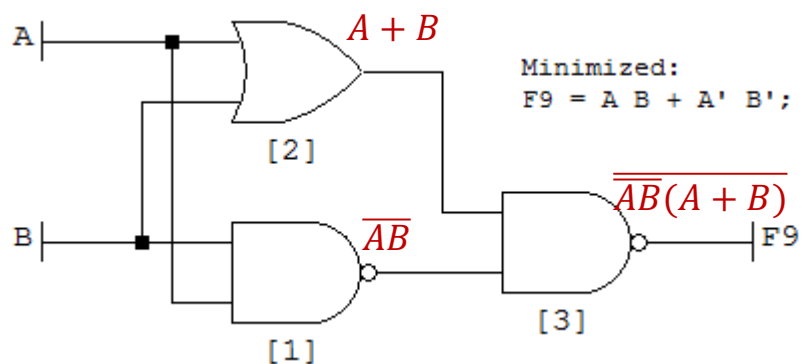


如果有多个逻辑变量输入时，如 $F = A \oplus B \oplus C$ ，输入变量中有奇数个1时，异或门输出为1。

A	B	C	$N = A \oplus B$	$Y = N \oplus C = A \oplus B \oplus C$
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

XNOR GATE CIRCUIT CONFIGURATIONS

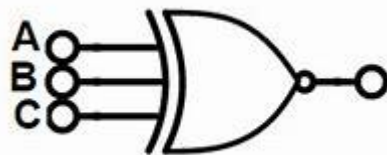
EXCLUSIVE NOR (同或门)



$$\overline{A \oplus B} = A \odot B$$

$$A \oplus B \oplus C = A \odot B \odot C$$

与前一页的表格比较得出结果相同



A	B	C	M = A ⊙ B	F = M ⊙ C = A ⊙ B ⊙ C
0	0	0	1	0
0	0	1	1	1
0	1	0	0	1
0	1	1	0	0
1	0	0	0	1
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

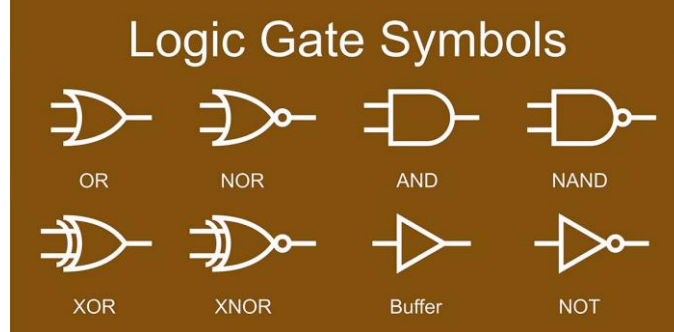
如果有偶数个逻辑变量输入时，如 $F = A \odot B \odot C \odot D$,

① 偶数个输入变量中有偶数个1时，同或门输出为1。

② 偶数个输入变量中有奇数个1时，异或门输出为1。

如果有奇数个逻辑变量输入时，
(2n+1)变量异或和(2n+1)变量同或是相同的关系。
当多输入变量中有奇数个1时，异或门输出为1。

内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

本章小结

逻辑代数的基本定律

- 德·摩根定律 (De Morgan's laws)
 - 在数理逻辑的定理推演中, 在计算机的逻辑设计中以及数学的集合运算中都起着重要的作用。
 - 他的发现影响了乔治·布尔从事的逻辑问题代数解法的研究。

表达式表示的为图片中的



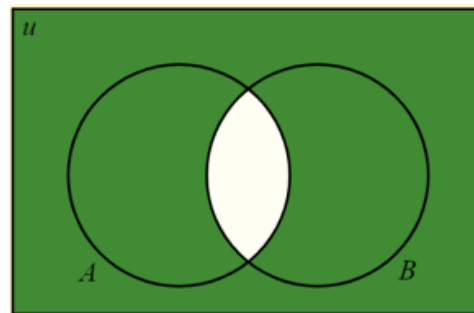
色区域

- 德·摩根定律/定理, 亦称反演律

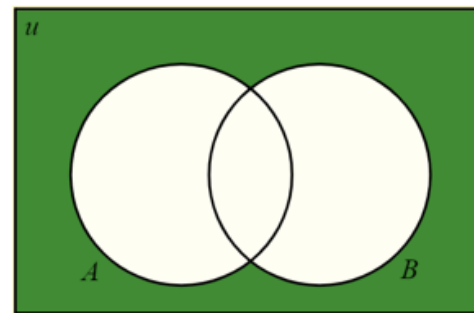
- $\overline{A \cdot B} = \bar{A} + \bar{B}$

- $\overline{A + B} = \bar{A} \cdot \bar{B}$

De Morgan's Law



$$\overline{A \cap B} = \bar{A} \cup \bar{B}$$



$$\overline{A \cup B} = \bar{A} \cap \bar{B}$$

常用的逻辑代数基本定律

Law/Theorem	Law of Multiplication	Law of Addition
交换律Commutative Law	$AB = BA$	$A + B = B + A$
结合律Associative Law	$A(BC) = (AB)C$	$A + (B + C) = (A + B) + C$
分配律Distributive Law	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
吸收律Absorption Law	$A(A + B) = A$	$A + AB = A$
控制律Dominant Law	$A \cdot 0 = 0$	$A + 1 = 1$
自等率Identity Law	$A \cdot 1 = A$	$A + 0 = A$
重叠律Idempotent Law	$A \cdot A = A$	$A + A = A$
互补律Complement Law	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
双重否定律Involution Law	$\overline{\overline{A}} = A$	$\overline{\overline{A}} = A$
反演律De Morgan's Law	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

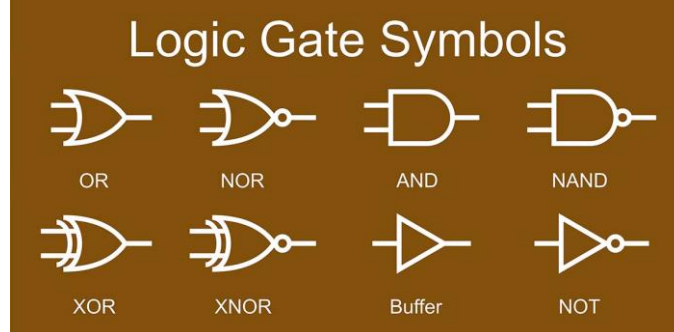
逻辑运算的完备集

- **完备集**的定义：一个代数系统，如果仅用它所定义的运算中的**某一组运算就能实现所有的运算**，则这一组运算是完备的，称为完备集。
- 运算集合**{与、或、非}**是完备集
 - 根据逻辑代数系统的定义，任何复杂的逻辑运算都是由与、或、非三中基本逻辑运算组合运用实现的。
- 运算集合**{或、非}**也是完备集
 - $AB = \overline{\overline{A} \overline{B}} = \overline{\overline{A} + \overline{B}}$
- 运算集合**{与、非}**也是完备集
 - $A + B = \overline{\overline{A} \overline{B}} = \overline{\overline{A} \cdot \overline{B}}$
- **{与、或}不是完备集**

至少要有“非”运算



内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

本章小结

代换规则 (REPLACEMENT)

- 任何含有 X 的逻辑等式，若将所有出现 X 的地方都用另一逻辑表达式 Y 代换，则等式仍然成立。
 - 可以用来扩展逻辑代数基本定律的应用。

例1：证明反演律推论 $\overline{A + B + C} = \bar{A} \cdot \bar{B} \cdot \bar{C}$
 根据反演律 $\overline{A + G} = \bar{A} \cdot \bar{G}$ ，设 $G = B + C$

$$\therefore \overline{A + (B + C)} = \bar{A} \cdot \overline{B + C} = \bar{A} \cdot \bar{B} \cdot \bar{C}$$

得证。

例2：证明 $A + ABC = A$
 根据吸收律 $A + AG = A$ ，设 $G = BC$

$$\therefore A + ABC = A$$

得证。

对偶规则 (DUAL)

$$F \neq F^*$$



○ 对任一逻辑函数 $F = f(X_1, X_2, \dots, X_n)$,
将表达式中所有的

- 逻辑常量1与0对换、
- 逻辑符号+与 $\cdot$ 对换,
- 保持原函数变量之间的运算顺序不变,

得到的新函数就是原函数 F 的对偶函数 F^*

- 原函数所具有的一切性质, 其对偶函数同样具备。若两个函数 $F = G$ 相等 $\rightarrow F^* = G^*$ (对偶函数也相等)
- 第17页幻灯片中表格两边的逻辑函数相互对偶
- 对偶函数 F^* 与原函数 F 是完全不同的函数, 只是形式上对偶。

应用对偶规则的注意点:

- 必须对所有的逻辑常量、逻辑符号进行变换;
- 必须保持运算顺序, 必要时可使用括号

对偶规则应用实例

- 证明反演律推论 $\overline{A \cdot B \cdot C} = \bar{A} + \bar{B} + \bar{C}$
 - 根据第20页幻灯片的反演律推论 $\overline{A + B + C} = \bar{A} \cdot \bar{B} \cdot \bar{C}$
 - 设 $F = \overline{A + B + C}$, 则 $F^* = \overline{A \cdot B \cdot C}$
 - 设 $G = \bar{A} \cdot \bar{B} \cdot \bar{C}$, 则 $G^* = \bar{A} + \bar{B} + \bar{C}$
 - $F = G \rightarrow F^* = G^*$
 - $\therefore \quad \overline{A \cdot B \cdot C} = \bar{A} + \bar{B} + \bar{C}$
 - 得证

This is an example ◻

反演规则 (INVERT)

$$F + \bar{F} = 1$$

○ 对逻辑函数 $F = f(X_1, X_2, \dots, X_n)$ ，只要将表达式中所有的

- 逻辑常量1与0对换、
- 逻辑符号+与·对换，
- 逻辑变量 $\bar{X}_i$ 与 X_i 之间互换，
- 保持原函数变量之间的运算顺序不变，

得到的新函数就是原函数 F 的反函数 $\bar{F}$ 。

- 反演规则，也称求反或者求补规则
- 反演规则是反演律的推广
- 反演规则主要用来求取逻辑函数的反函数。

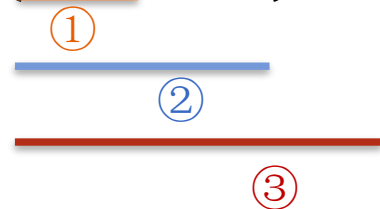
应用反演规则的注意点：

- 必须对所有的逻辑常量、逻辑符号和逻辑变量进行变换；
- 必须保持运算顺序，必要时可使用括号
- 区分非运算和反变量

$\bar{X}_i$ 与 X_i 之间的互换仅对变量成立，非运算须保留。

反演规则应用实例1

- 求 $F = (A \cdot \bar{B} + C) \cdot D$ 的反函数 $\bar{F}$



- 根据反演规则直接得出反函数 $\bar{F} = (\bar{A} + B) \cdot \bar{C} + \bar{D}$
- 使用反演律证明如下:

$$\bar{F} = \overline{(A \cdot \bar{B} + C) \cdot D} = \overline{A \cdot \bar{B} + C} + \bar{D} = \overline{A \cdot \bar{B}} \cdot \bar{C} + \bar{D} = (\bar{A} + B) \cdot \bar{C} + \bar{D}$$

反演规则应用实例2

- 求 $F = \overline{AB} + \bar{C}(D + A)$ 的反函数 $\bar{F}$

非运算，
不是反变量

- 根据反演规则，非运算须保留

直接得出表达式

进一步化简

$$\bar{F} = \overline{\overline{A} + \bar{B} \cdot (C + \bar{D} \cdot \bar{A})} = AB(C + \bar{D} \cdot \bar{A})$$

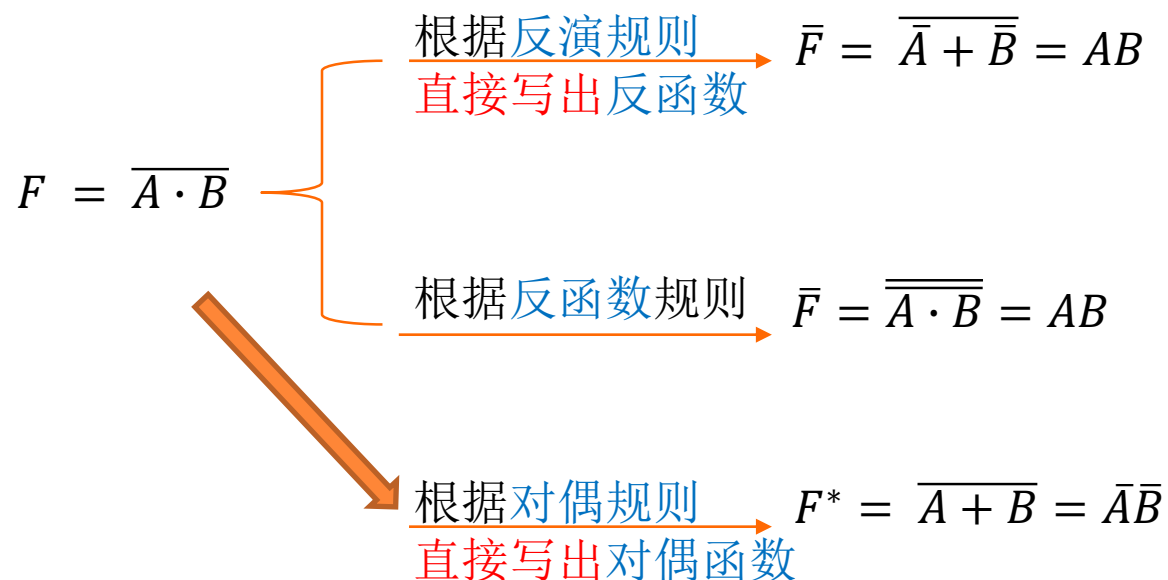
- 解法2:**

- 设 $G = AB$ ，将 G 看作一个整体变量
- 则 $F = \bar{G} + \bar{C}(D + A)$
- 根据反演规则 $\bar{F} = \overline{\bar{G} + \bar{C}(D + A)} = G \cdot (C + \bar{D} \cdot \bar{A})$
- 根据代换规则 $\bar{F} = AB(C + \bar{D} \cdot \bar{A})$

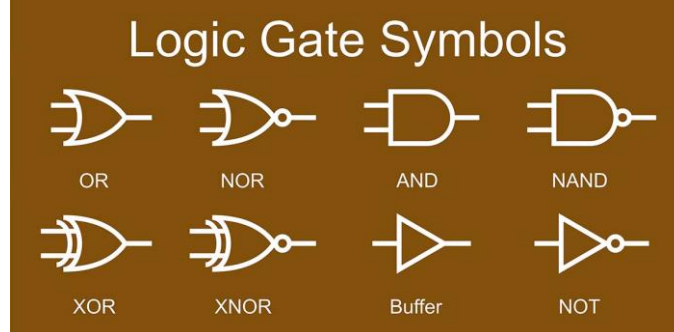
注意
运算顺序

对偶规则与反演规则的比较和总结

	对偶规则	反演规则
常量1与0对换		
逻辑符号+与·对换		
逻辑变量 X_i 与 $\bar{X}_i$ 之间互换		
保持原函数变量之间的运算顺序不变		



内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

本章小结

并项公式

两个与项
逻辑相邻

逻辑表达式中的两个与项，除了一个变量因子互补外，其余的变量因子都相同，则称这两个与项是“相邻”（逻辑相邻）的。

$$\bigcirc AB + \bigcirc \bar{A}B = B$$

• 证明：

$$\begin{aligned} AB + \bar{A}B &= (A + \bar{A})B \\ &= 1 \cdot B \\ &= B \end{aligned}$$

分配律反向应用

互补律

自等律

• 应用代换规则，将变量 B 用与项 BC 替换

$$ABC + \bar{A}BC = BC$$

• 根据对偶规则，则有推论

$$\bigcirc (A + B) \cdot \bigcirc (\bar{A} + B) = B$$

两个或项
逻辑相邻

并项公式含义：

两个相邻项可以合并为一项，
消去一个互补的变量。

消冗余因子公式

○ $A + \bar{A} \cdot B = A + B$

• 证明:

$$\begin{aligned} A + \bar{A}B &= (A + \bar{A})(A + B) \\ &= 1 \cdot (A + B) \\ &= A + B \end{aligned}$$

分配律

互补律

自等律

○ 应用代换规则，将变量 B 用与项 BC 替换

$$A + \bar{A}BC = A + BC$$

○ 根据对偶规则，则有推论

$$A(\bar{A} + B) = A \cdot B$$

消冗余项公式

○ $AB + \bar{A}C + \textcircled{BC} = AB + \bar{A}C$

• 证明: $AB + \bar{A}C + BC = AB + \bar{A}C + (A + \bar{A})BC$
 $= AB + \bar{A}C + ABC + \bar{A}BC$
 $= AB(1 + C) + \bar{A}C(1 + B)$
 $= AB + \bar{A}C$

互补律

分配律

结合律、分配率

控制律

○ 推论: $AB + \bar{A}C + BCD = AB + \bar{A}C$

• 证明: $AB + \bar{A}C + BCD = \underline{AB + \bar{A}C + BC} + BCD$
 $= AB + \bar{A}C + (1 + D)BC$
 $= AB + \bar{A}C + BC$
 $= AB + \bar{A}C$

反用消冗余

分配率

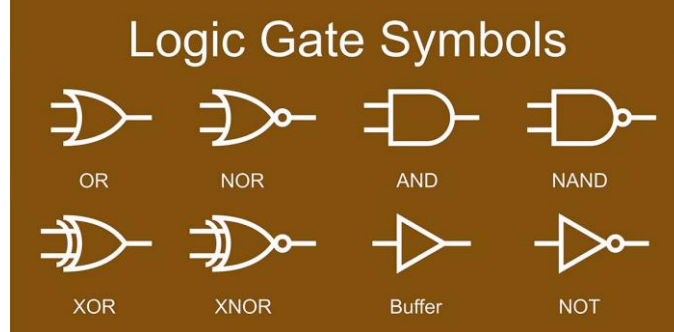
控制律

消冗余公式

○ 根据对偶规则, 则有推论

• $(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$

内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

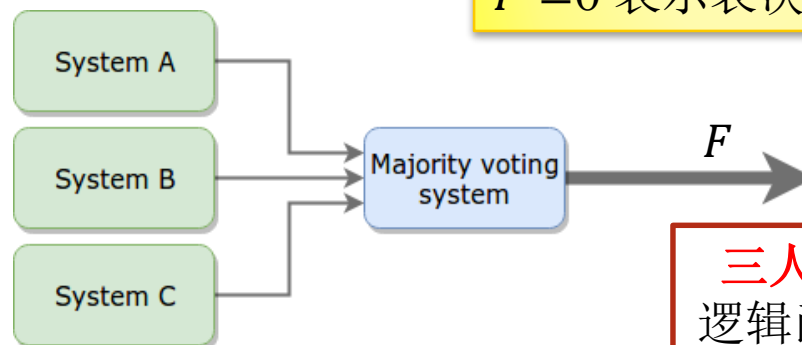
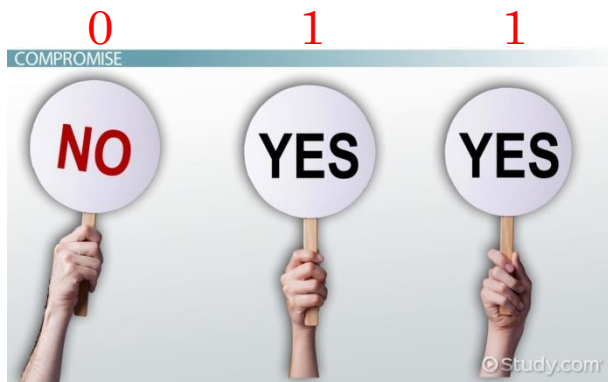
3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

本章小结

逻辑函数表达式 (不具有唯一性)

以下各式真值表结果相同



$F = 1$ 表示表决成功通过
 $F = 0$ 表示表决失败

三人表决器的
逻辑函数表达式

○ $F = f(A, B, C) = ABC\bar{C} + \bar{A}BC + A\bar{B}C + ABC$

○ $F = f(A, B, C) = AB + BC + AC + ABC$ ----- 简化表达式

$= AB + BC + AC$ ----- 吸收率 ----- 与或式

○ $F^* = (A + B)(B + C)(A + C)$ ----- 对偶 ----- 或与式

○ $F^* = \overline{\overline{F^*}} = \overline{\bar{A}\bar{B} + \bar{B}\bar{C} + \bar{A}\bar{C}}$ ----- 反演 ----- 与或非式

一般来说 $F \neq F^*$ ，这里 $F = F^*$ 时，逻辑功能仍然正确。

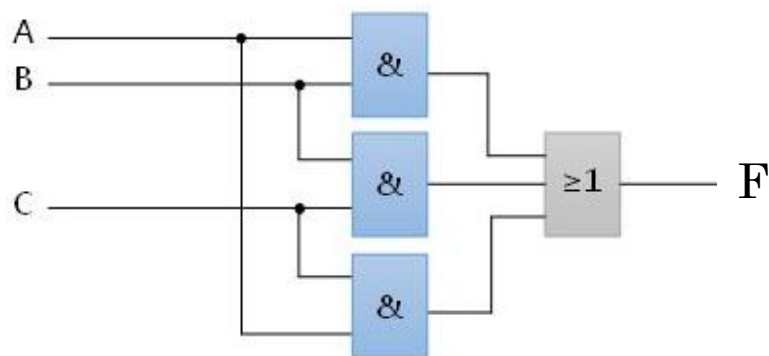
非两个以上
裁判说no

逻辑图（不具有唯一性）

- 每个逻辑表达式都可以用逻辑符号连接成的逻辑图表示。

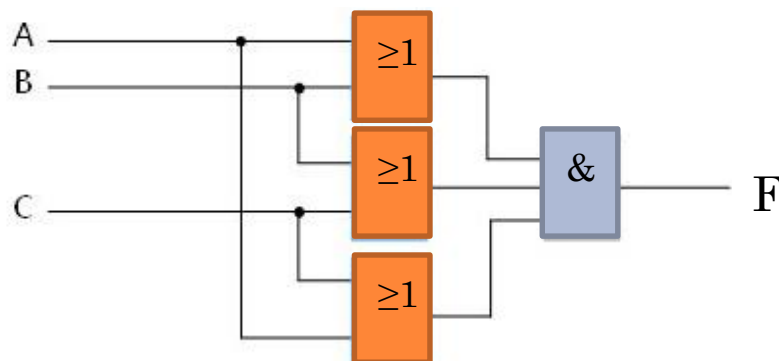
实际手动画图不用仿照打印体，可以按照教科书P67的样子来画。

$$F = AB + BC + AC$$

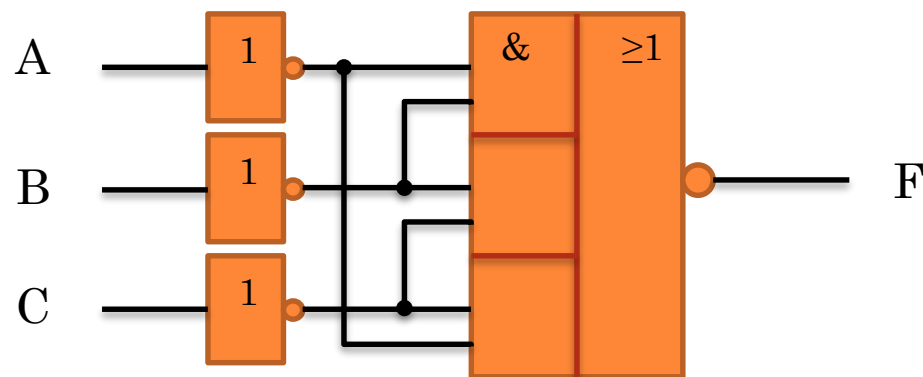


两级AND-OR 或 NAND 门电路表示“积的和”

$$F^* = (A + B)(B + C)(A + C)$$



两级 OR-AND 或 NOR 门电路表示“和的积”



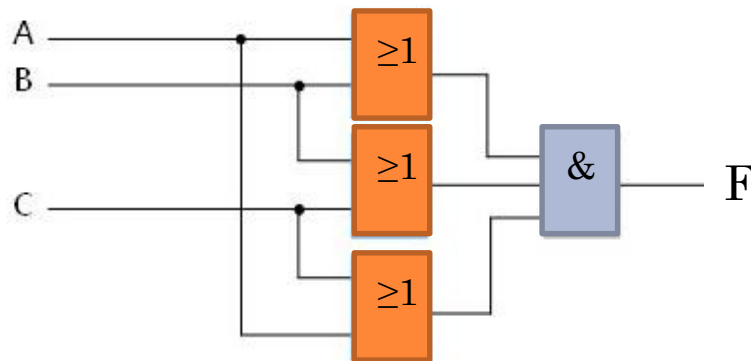
不同逻辑图描述
同一个逻辑功能

$$F^* = \overline{A}\overline{B} + \overline{B}\overline{C} + \overline{A}\overline{C}$$

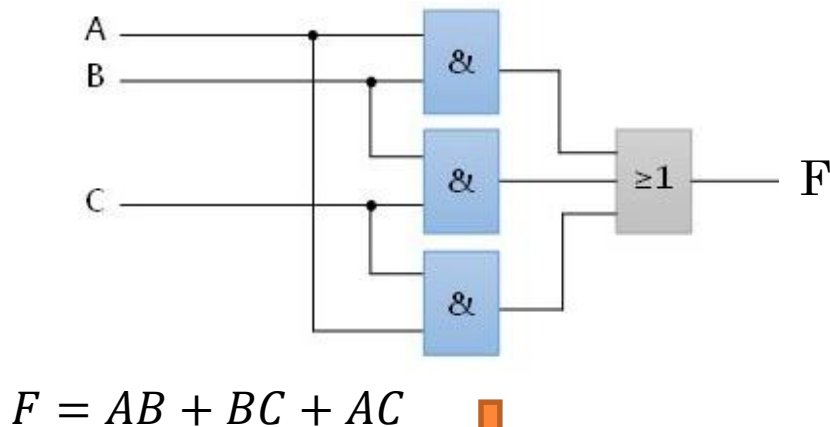
真值表 (具有唯一性)

- 每个逻辑函数都对应有一张真值表 (truth table)
- 真值表穷举了函数的所有情况

同样的真值表也说明三人表决器的逻辑函数表达式 $F = F^*$



$$F^* = (A + B)(B + C)(A + C)$$



Truth table

A	B	C	OUT
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

坐标按一维方式排列

卡诺图 KARNAUGH MAP (具有唯一性)

回忆：若任意两个相邻的代码只有一位二进制数不同，最大数与最小数之间也仅有一位数不同，首尾相连，称为“**循环码**”

	$\bar{C}$	C
$\bar{A} \cdot \bar{B}$	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	$\bar{A} \cdot \bar{B} \cdot C$
$\bar{A} \cdot B$	$\bar{A} \cdot B \cdot \bar{C}$	$\bar{A} \cdot B \cdot C$
$A \cdot B$	$A \cdot B \cdot \bar{C}$	$A \cdot B \cdot C$
$A \cdot \bar{B}$	$A \cdot \bar{B} \cdot \bar{C}$	$A \cdot \bar{B} \cdot C$



$AB \backslash C$	0	1
00	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	$\bar{A} \cdot \bar{B} \cdot C$
01	$\bar{A} \cdot B \cdot \bar{C}$	$\bar{A} \cdot B \cdot C$
11	$A \cdot B \cdot \bar{C}$	$A \cdot B \cdot C$
10	$A \cdot \bar{B} \cdot \bar{C}$	$A \cdot \bar{B} \cdot C$

循环码

$A \backslash BC$	00	01	11	10
0			1	
1		1	1	1

取值为**1**的变量在**最小项**中写为**原变量X**，
取值为**0**的变量在**最小项**中写为**反变量 $\bar{X}$** 。

$AB \backslash C$	0	1
00		
01		1
11	1	1
10		1

坐标按**二维**方式排列

三人表决器对应的卡诺图，通常函数 $F = 1$ 的方块内填**1**，而函数 $F = 0$ 的方块内不填数字或者填0。

标准表达式

- 前面几种逻辑函数描述方式中，真值表和卡诺图对于每个函数是唯一的
 - 判断两个函数是否相等，只要看其真值表或卡诺图是否相同。
- 逻辑表达式和逻辑图描述逻辑函数没有唯一性，不能判断相等问题。
 - 逻辑函数常常描述为与或表达式和或与表达式
 - 与或（AND-OR）表达式，亦称为积之和（sum of product）表达式，例如
$$F = ABC + DE$$
 - 或与（OR-AND）表达式，亦称为和之积（product of sum）表达式，例如
$$F = (A+B+C)(D+E)$$

最小项和标准与-或表达式

具有唯一性

- 真值表与卡诺图用穷举法描述了逻辑函数
 - 真值表中的一行（或卡诺图中的一块）代表着一个与项关系，通常这种与项成为函数的最小项（minterm）
- 最小项：在 n 变量逻辑函数中，若 m 为包含 n 个因子的乘积项，而且这 n 个变量均以原变量或反变量的形式在 m 中出现一次，则称 m 为该组变量的最小项。
 - 一个 n 变量逻辑函数共有 2^n 个最小项，每个最小项是 n 个变量构成的与项。
 - 例如， A, B, C 三个变量的最小项共有 $2^3 = 8$ 个：
 $\bar{A}\bar{B}\bar{C}, \bar{A}\bar{B}C, \bar{A}B\bar{C}, \bar{A}BC, A\bar{B}\bar{C}, A\bar{B}C, AB\bar{C}, ABC$
- 最小项的重要性质：
 - 在输入变量的任何取值下必有一个最小项，而且仅有一个最小项的值为1
 - 全体最小项之和为1
 - 任意两个最小项的乘积为0
 - 具有相邻性的两个最小项之和可以合并成一项并消去一对因子
 $\bar{A}B\bar{C} + AB\bar{C} = (\bar{A} + A)B\bar{C} = B\bar{C}$ （根据并项公式）
- 标准与-或表达式： $F = \sum_0^{2^n-1} f_i \cdot m_i$ (m_i 是一个与项)

最大项和标准或-与表达式

具有唯一性

- $\bar{F} = \sum_0^{2^n-1} \bar{f}_i \cdot m_i$ (从卡诺图上看, 反函数 $0 \rightarrow 1, 1 \rightarrow 0, \therefore f_i \rightarrow \bar{f}_i$)
- $F = \bar{\bar{F}} = \overline{\sum_0^{2^n-1} \bar{f}_i \cdot m_i} = \prod_0^{2^n-1} (\bar{\bar{f}}_i + \bar{m}_i) = \prod_0^{2^n-1} (f_i + M_i)$
- $M_i = \bar{m}_i$ 是一个或项, 称为最大项。反演律
- 最大项**: 在 n 变量逻辑函数中, 若 M 为 n 个变量之和, 而且这 n 个变量均以原变量或反变量的形式在 M 中出现一次, 则称 M 为该组变量的最大项。
- 最大项的主要性质:
 - 在输入变量的任何取值下必有一个最大项, 而且只有一个最大项的值为 0
 - 全体最大项之积为 0
 - 任意两个最大项之和为 1
 - 只有一个变量不同的两个最大项的乘积等于各相同变量之和。
- 一个逻辑函数可以表示为若干最大项的乘积, 与最小项之和表达式一样, 最大项之积表达式也是唯一的。

$$(A + G)(\bar{A} + G) = G, \quad \text{若 } G = \bar{B} + C$$

$$(A + \bar{B} + C)(\bar{A} + \bar{B} + C) = \bar{B} + C$$

分配律

三人表决器的标准表达式

$\begin{array}{c} BC \\ \backslash A \end{array}$	00	01	11	10
0			1	
1		1	1	1

- 三人表决器的标准与-或表达式
- $$F = 0 \cdot \bar{A}\bar{B}\bar{C} + 0 \cdot \bar{A}\bar{B}C + 0 \cdot \bar{A}B\bar{C} + 1 \cdot \bar{A}BC + 0 \cdot A\bar{B}\bar{C} + 1 \cdot A\bar{B}C + 1 \cdot AB\bar{C} + 1 \cdot ABC$$

$$= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC = m_3 + m_5 + m_6 + m_7$$
- 简写为 $F = \sum m(3,5,6,7)$
- 三人表决器的标准或-与表达式
- $$F = (0 + A + B + C)(0 + A + B + \bar{C})(0 + A + \bar{B} + C)(1 + A + \bar{B} + \bar{C})(0 + \bar{A} + B + C)(1 + \bar{A} + B + \bar{C})(1 + \bar{A} + \bar{B} + C)(1 + \bar{A} + \bar{B} + \bar{C}) = M_0 \cdot M_1 \cdot M_2 \cdot M_4$$
- 简写为 $F = \prod M(0,1,2,4)$

三变量函数的最小项和最大项

$$M_i = \bar{m}_i$$

Row No.	A	B	C	minTerm	maxTerm	Value of Function F
0	0	0	0	$\bar{A} \cdot \bar{B} \cdot \bar{C} = m_0$	$A + B + C = M_0$	f_0
1	0	0	1	$\bar{A} \cdot \bar{B} \cdot C = m_1$	$A + B + \bar{C} = M_1$	f_1
2	0	1	0	$\bar{A} \cdot B \cdot \bar{C} = m_2$	$A + \bar{B} + C = M_2$	f_2
3	0	1	1	$\bar{A} \cdot B \cdot C = m_3$	$A + \bar{B} + \bar{C} = M_3$	f_3
4	1	0	0	$A \cdot \bar{B} \cdot \bar{C} = m_4$	$\bar{A} + B + C = M_4$	f_4
5	1	0	1	$A \cdot \bar{B} \cdot C = m_5$	$\bar{A} + B + \bar{C} = M_5$	f_5
6	1	1	0	$A \cdot B \cdot \bar{C} = m_6$	$\bar{A} + \bar{B} + C = M_6$	f_6
7	1	1	1	$A \cdot B \cdot C = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$	f_7



注意：在最小项和最大项中，原变量和反变量代表的0和1不一样

最小项与卡诺图的关系

Three-variable Karnaugh map
Filled with **minterm** numbers

A \ BC	BC			
	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

AB \ C	0	1
00	m_0	m_1
01	m_2	m_3
11	m_6	m_7
10	m_4	m_5

Four-variable Karnaugh map
Filled with **minterm** numbers

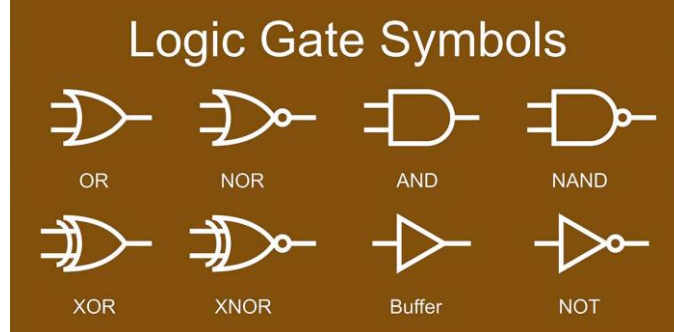
AB \ CD	CD			
	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

卡诺图坐标采用**循环码**编排

几何位置相邻的两个最小项
仅有一个变量是不同的，以
保证这两个最小项**逻辑上也**
相邻。

处于任何一行或者一列
两端的最小项也仅有一个变量不同，所以它们
也具有**逻辑相邻性**。

内容



○ 3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

○ 3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

○ 3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

○ 3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

○ 3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

○ 3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

○ 3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

○ 本章小结

逻辑简化的意义和标准

- 一个逻辑函数可以有多种表达式，从而有相对应的繁简不一的逻辑电路。
- 实现逻辑电路的门电路个数不同，每个门电路输入变量数不同，造成集成电路的资源消耗也不同。
- 逻辑函数的化简会减少器件使用，减少芯片面积。



然而，采用中规模集成电路或者大规模集成电路实现逻辑电路时，化简的意义并不明显。

公式法简化

- 需要对公式使用比较熟练，并掌握一定技巧。
- 例：简化逻辑函数，求最简与-或式

$$\begin{aligned}
 f(A, B, C, D) &= \bar{A}\bar{C}\bar{D} + \bar{B}\bar{C}\bar{D} + A\bar{B}D + \bar{B}CD \\
 &= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + \\
 &\quad A\bar{B}\bar{C}D + A\bar{B}CD + \bar{A}\bar{B}CD + A\bar{B}CD \\
 &= \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}D + \\
 &\quad A\bar{B}CD + \bar{A}\bar{B}CD \\
 &= \bar{A}\bar{C}\bar{D} + A\bar{B}\bar{C} + \bar{B}CD
 \end{aligned}$$

重叠律

$$A + A = A$$

并项公式

$$AB + \bar{A}B = B$$

卡诺图法简化

(1) 化简为最简与-或式

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
1	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	BC	$B\bar{C}$
1	$A\bar{B}\bar{C}$	$A\bar{B}C$		

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	B	
1	$A\bar{B}\bar{C}$	$A\bar{B}C$		

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
1	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

$\bar{A}\bar{C}$

$\begin{array}{c} BC \\ \diagdown \\ A \end{array}$	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
1	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

$\bar{C}$

例1: 简化 $F = A\bar{C} + \bar{A}C + B\bar{C} + \bar{B}C$
求最简与-或式

$A \backslash BC$	00	01	11	10
0		1	1	1
1	1	1		1

$$F = A\bar{B} + \bar{A}C + B\bar{C}$$

$A \backslash BC$	00	01	11	10
0		1	1	1
1	1	1		1

$$F = A\bar{C} + \bar{B}C + \bar{A}B$$

两个化简结果都符合最简与或式的标准



有时一个逻辑函数的化简结果不是唯一的。

例2: 简化 $F = f(A, B, C, D) = \sum m(0, 1, 2, 4, 5, 7, 9, 12)$
求最简与-或式

$\begin{array}{c} CD \\ \backslash AB \end{array}$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

$\begin{array}{c} CD \\ \backslash AB \end{array}$	00	01	11	10
00	1	1		1
01	1	1	1	
11	1			
10		1		

得到最简与-或式 $F = \bar{A}\bar{B}\bar{D} + \bar{A}BD + B\bar{C}\bar{D} + \bar{B}\bar{C}D$

- 在合并最小项时，应首先考虑那些只有一个合并可能的最小项，当这些最小项都被圈过以后，再对那些剩余的最小项圈并。
- 每个圈中都要有一个没有被圈过的最小项
 - 蓝色的卡诺圈虽然最大，但其所包含的4个最小项已被其它的圈都圈过了，因而所对应项 $\bar{A}\bar{C}$ 是冗余的。

例3: 简化 $F = ABC + ABD + A\bar{C}D + \bar{C}\bar{D} + A\bar{B}C + \bar{A}C\bar{D}$
求最简与-或式

$AB \backslash CD$	00	01	11	10
00	1			1
01	1			1
11	1	1	1	1
10	1	1	1	1

最简式为 $F = A + \bar{D}$

$AB \backslash CD$	00	01	11	10
00	1			1
01	1			1
11	1	1	1	1
10	1	1	1	1

$\bar{F} = \bar{A}D$

$F = \bar{\bar{F}} = \overline{\bar{A}D} = A + \bar{D}$

- 卡诺图中卡诺圈的个数越少越简
- 卡诺图中卡诺圈内的最小项越多（即圈越大）越简
- 前面的例子都是通过合并卡诺图中的1来求化简结果的。

- 全部最小项之和为1, $F + \bar{F} = 1$
- 当卡诺图中0的数目远小于1的数目时，合并0的方法更简便。
- 需要化简为或与式时，合并0的方式更合适。
- 如果要求 $\bar{F}$ 的化简结果，合并0的方式更简便。

卡诺图法简化

(2) 化简为最简或-与式

- 若要求取逻辑函数最简或-与式，则必须在卡诺图上圈出最大项（maxTerm），而最大项对应于卡诺图上真值为0的方格。

$AB \backslash CD$	00	01	11	10
00	1		1	
01	1		1	1
11	1		1	1
10		1		

最简与-或式

$$F = BC + B\bar{D} + \bar{A}\bar{C}\bar{D} + \bar{A}CD + A\bar{B}\bar{C}D$$

$AB \backslash CD$	00	01	11	10
00	1		1	
01	1		1	1
11	1		1	1
10		1		

最简或-与式

$$F = (A + C + \bar{D})(B + \bar{C} + D)(\bar{B} + C + \bar{D})(\bar{A} + B + D)(\bar{A} + B + \bar{C})$$

非完全定义逻辑函数

- 完全定义逻辑函数：逻辑变量的所有可能的取值组合都有定义。
- 在实际中，受到应用背景或约束条件的影响，经常会有某些逻辑变量的组合不能或不会出现，输入组合所对应的函数输出就不需要进行定义，这种逻辑函数称之为非完全定义的逻辑函数。
 - 例：十进制数用NBCD码（Natural Binary-coded Decimal）表示（1位十进制的NBCD码包含4位二进制数位），函数 F 代表对1位十进制数“四舍五入”的进位信号，该函数的真值表如右表所示。
 - 十进制运算中只有前10个数码有定义，10~15这6个数码无定义，在真值表和卡诺图上填写 x （或者 $-$ 、 d 或 Φ ）

十进制	B_8	B_4	B_2	B_1	F
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	x
11	1	0	1	1	x
12	1	1	0	0	x
13	1	1	0	1	x
14	1	1	1	0	x
15	1	1	1	1	x

非完全定义逻辑函数的简化

- 四舍五入函数的标准与-或表达式

$$F = \sum m(5,6,7,8,9) + \sum d(10,11,12,13,14,15), \quad \sum d(\dots) = 0$$

- 四舍五入函数的标准或-与表达式

$$F = \prod M(0,1,2,3,4) \cdot \prod d(10,11,12,13,14,15), \quad \prod d(\dots) = 1$$

- 这些无关项可任意定义为1或0,
 - 划入卡诺圈的定义为1, 未划入卡诺圈的定义为0, 以便消去更多变量。

$B_2B_1 \backslash B_8B_4$		B_2B_1			
		00	01	11	10
00	m_0	m_0	m_1	m_3	m_2
01	m_4	m_4	m_5	m_7	m_6
11	m_{12}	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_8	m_9	m_{11}	m_{10}

$B_2B_1 \backslash B_8B_4$		B_2B_1			
		00	01	11	10
00		0	0	0	0
01		0	1	1	1
11		x	x	x	x
10		1	1	x	x

最简与-或式和约束条件

$$\left\{ \begin{array}{l} F = B_8 + B_4B_2 + B_4B_1 \\ B_8B_4 + B_8B_2 = 0 \end{array} \right.$$

通过卡诺图得出约束条件

约束项、任意项和无关项

(参考《清华教材》P47~50)

- **约束项**：不可能/不允许出现的输入变量组合
 - 每个约束项对应的最小项的值恒等于0
 - 可以将约束项写进逻辑函数或者从中删除，不影响函数值。
 - 使约束项取值为1的输入变量是不允许出现的。
- **任意项**：在输入变量的某些取值下函数值是1还是0皆可，并不影响电路的功能。在这些变量取值下，其值为1的那些最小项成为任意项。
 - 存在任意项的逻辑函数也叫做不完全定义的逻辑函数。
 - 有可能出现使任意项取值为1的输入变量取值。
- **无关项**：约束项和任意项的统称。
 - “无关”指是否把这些最小项写入逻辑函数式无关紧要，可以写入也可以删除。

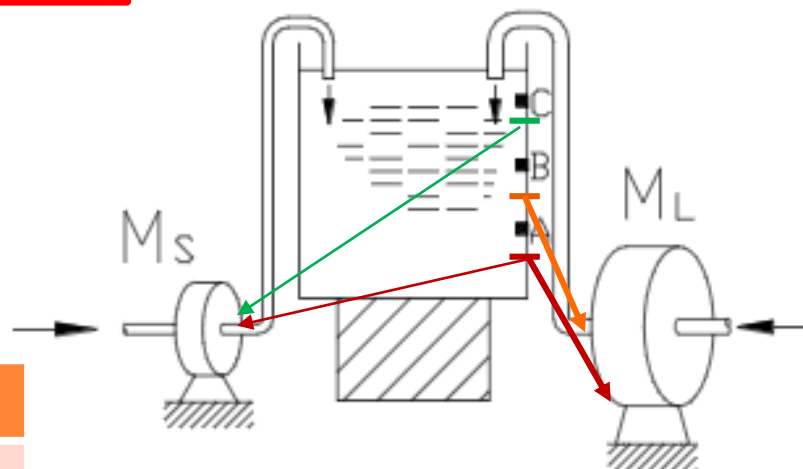


有一水箱，由大、小两台补水水泵 M_L 和 M_S 供水，如图所示。水箱中设置了 3 个水位检测元件 A、B、C。水面低于检测元件时，检测元件给出 低电平 水面高于检测元件时，检测元件给出 高电平 现要求当水位超过 C 点时水泵停止工作；水位低于 C 点而高于 B 点时 M_S 单独工作；水位低于 B 点而高于 A 点时 M_L 单独工作；水位低于 A 点时 M_L 和 M_S 同时工作。

设：高电平用1表示，低电平用0表示

- Y_L 和 Y_S 分别表示 M_L 和 M_S 的启动控制信号，
- $Y_L, Y_S = 1$ ，水泵启动；
- $Y_L, Y_S = 0$ ，水泵停止；
- 水位为 L_W

情形描述	A	B	C	Y_L	Y_S
$L_W > C$	1	1	1	0	0
$B < L_W < C$	1	1	0	0	1
$A < L_W < B$	1	0	0	1	0
$L_W < A$	0	0	0	1	1



$$Y_L = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} = \bar{B}\bar{C}$$

$$Y_S = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C}$$

$$ABC \neq 001, 010, 011, 101$$

每个约束项 $\equiv 0$,

$$\bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C = 0$$

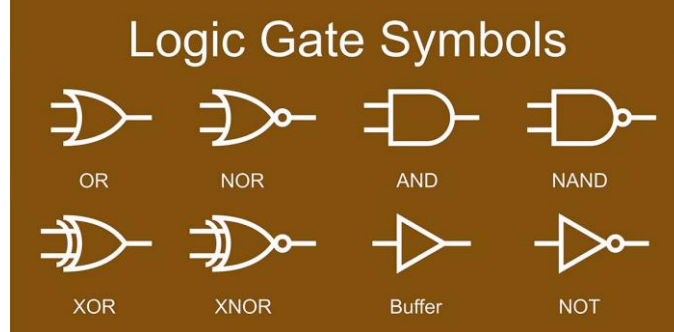
任意项的物理概念 (一个电动机控制的例子)

- 现以三个逻辑变量A、B、C分别表示一台电动机的正转、反转和停止的命令。则表示正转、反转和停止工作状态的逻辑函数可写成

逻辑变量取值	说明		
A=1	正转	$Y_1 = A\bar{B}\bar{C}$	(正转)
B=1	反转	$Y_2 = \bar{A}B\bar{C}$	(反转)
C=1	停止	$Y_3 = \bar{A}\bar{B}C$	(停止)

- 注意：**任何时刻，电动机只能执行其中一个命令，所以A、B、C当中出现两个以上为1时，电动机将无法工作。
- 实际的电路设计：**当A、B、C三个控制变量出现两个以上同时为1或者全部为0时，电路能自动切断供电电源。
 - 此时， $Y_1, Y_2, Y_3 = 1$ 或 0 无关紧要，电动机将会受到保护而停止运行。
 - 当 $A = B = C = 1$ 时，对应的 $ABC(m_7) = 1$ ，若将 m_7 写入 Y_1 ， $Y_1 = 1$ ；若不将 m_7 写入 Y_1 ， $Y_1 = 0$ 。 $Y_1 = 1$ 或者 0都是允许的。
 - 因此， ABC 、 $\bar{A}\bar{B}\bar{C}$ 、 $\bar{A}BC$ 、 $A\bar{B}C$ 、 ABC 都是 Y_1, Y_2, Y_3 的任意项。

内容



3.1 基本逻辑运算

- 逻辑代数的二值逻辑
- 逻辑非、乘、加和非、与、或运算
- 逻辑运算的优先级

3.2 复合逻辑运算

- 与非门、或非门、异或门、同或门
- 其它复合门

3.3 逻辑代数的基本定律

- 逻辑等式的证明
- 常用的基本定理
- 逻辑运算的完备集合

3.4 逻辑代数的基本规则

- 代换规则
- 对偶规则
- 反演规则

3.5 逻辑代数的常用公式

- 并项公式
- 消冗余因子公式
- 消冗余项公式

3.6 逻辑函数及其描述方法

- 逻辑函数表达式
- 逻辑图
- 真值表
- 卡诺图
- 标准表达式

3.7 逻辑函数的简化

- 逻辑简化的意义和标准
- 公式法简化
- 卡诺图法简化

本章小结



本章小结

- 逻辑函数的建立、表示和化简是逻辑函数分析和设计的基础，它的数学工具就是逻辑代数。
- 同一个逻辑函数可有 []、[]、[]、[] 四种不同形式来表示，
 - 其中真值表和卡诺图的表示方法是唯一的，逻辑函数的标准表达式也是唯一的。
- 两类逻辑函数的化简方法
 - 代数法化简
 - 卡诺图化简法